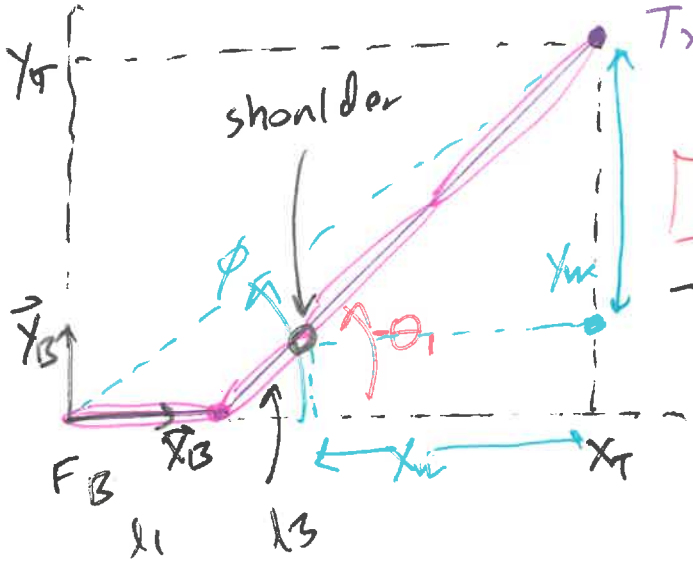
1) Find θ_1 - look from above



$\phi = \text{atan2}(y_T, x_T)$ but

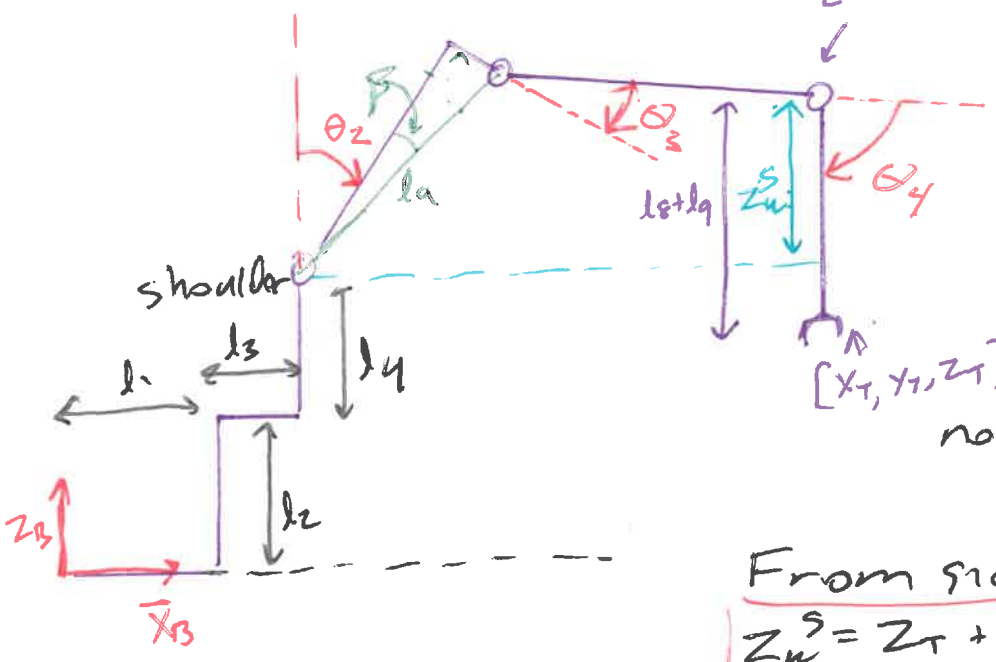
$$\theta_1 = -\text{atan2}(y_T, x_T - l_1)$$

- note the negative sign because $\vec{Z}_0$ points down

$\theta_1 = \text{atan2}(-y_T, x_T - l_1)$ is equivalent

---

2) Find wrist center wrt. shoulder



$$O_w^s = [x_w^s, y_w^s, z_w^s]$$

From top - down

$$x_w^s = x_T - l_1 - l_3 \cos(-\theta_1)$$
$$y_w^s = y_T - l_3 \sin(-\theta_1)$$

note $\cos(-\theta_1) = \cos(\theta_1)$

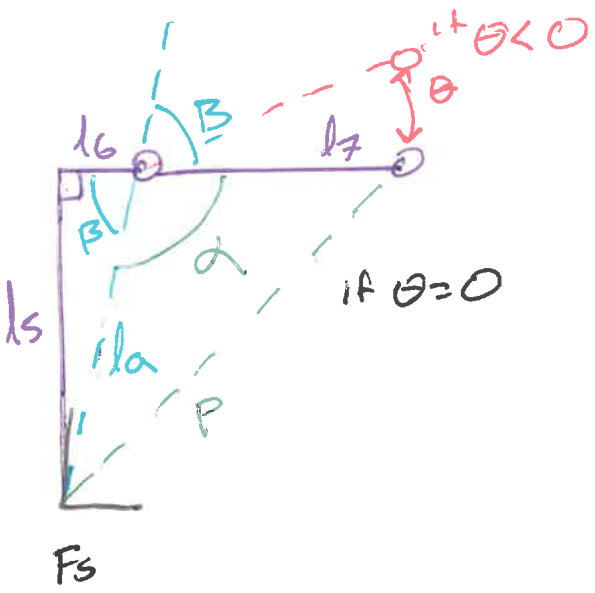
$\sin(-\theta_1) = -\sin(\theta_1)$

From side picture

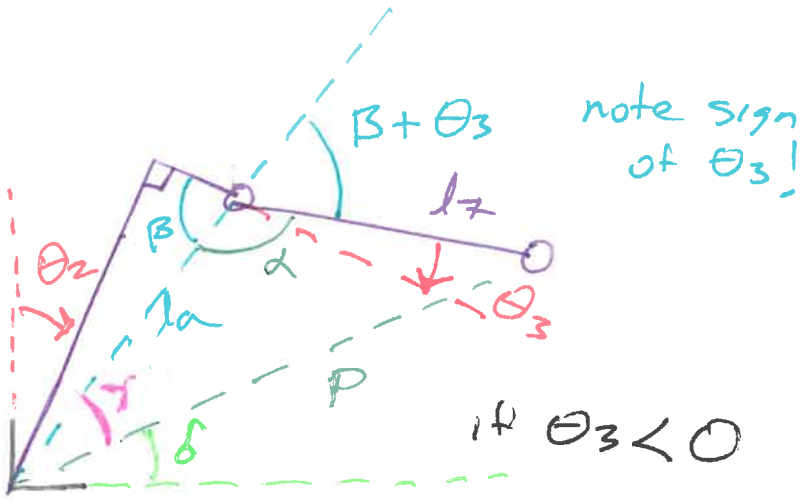
$$z_w^s = z_T + l_8 + l_9 - l_2 - l_4$$

3) $l_a = \sqrt{l_5^2 + l_6^2}$

$\beta = \text{atan}\left(\frac{l_6}{l_5}\right)$



4)



$\alpha = \pi - \beta$

$\alpha = \pi - \beta - \theta_3$

$\theta_3 = \pi - \beta - \alpha$

$P^2 = la^2 + l7^2 - 2\, la\, l7 \cos(\alpha)$

$$\alpha = \operatorname{acos}\left(\frac{P^2 - la^2 - l7^2}{-2\, la\, l7}\right)$$

or use $\operatorname{acos}(D) = \operatorname{atan2}(\sqrt{1-D^2}, D)$

5)

$$\gamma = \operatorname{acos}\left(\frac{la^2 + P^2 - l7^2}{2\, la\, P}\right)$$

$$\delta = \operatorname{atan2}\left(Z_w^s, \sqrt{X_w^{s\,2} + Y_w^{s\,2}}\right)$$

$\theta_2 = \pi - (\pi - \beta) - \gamma - \delta$

$\theta_2 = \beta - \gamma - \delta$

6) Looking at fig 2 we see

$\theta_4 = \pi/2 - \theta_2 - \theta_3$

```python
"""Instructor solution for Homework 3 inverse kinematics."""

import numpy as np


# Robot dimensions in meters
l1 = 0.0388353
l2 = 0.0624
l3 = 0.0303992
l4 = 0.0542
l5 = 0.11257
l6 = 0.0280
l7 = 0.1349
l8 = 0.0611
l9 = 0.1004


def inverse_kinematics(target):

    X, Y, Z = target

    # ------------------------------------------------------------
    # 1. Base rotation
    # ------------------------------------------------------------

    rho = np.sqrt((X - l1)**2 + Y**2)

    if rho < 1e-10:
        return {}, False

    theta_1 = -np.arctan2(Y, X - l1)


    # ------------------------------------------------------------
    # 2. Wrist center relative to the shoulder
    # ------------------------------------------------------------

    Xd = X - l1 - l3 * np.cos(theta_1)
    Yd = Y + l3 * np.sin(theta_1)

    Zd = Z + l8 + l9 - (l2 + l4)

    R = np.sqrt(Xd**2 + Yd**2)
    P = np.sqrt(R**2 + Zd**2)

    # ------------------------------------------------------------
    # 3. Effective upper-arm dimensions
    # ------------------------------------------------------------

    la = np.sqrt(l5**2 + l6**2)


    beta = np.arctan2(l5, l6)
    gamma = np.arctan2(l6, l5)

    # ------------------------------------------------------------
    # 4. Elbow angle
    # ------------------------------------------------------------

    #acos is fine, but cna throw a warning, atan2 is better
    # alpha = np.arccos(-(P**2 - la**2 - l7**2) / (2 * la * l7))
    D=-(P**2 - la**2 - l7**2) / (2 * la * l7)
    alpha = np.arctan2(np.sqrt(1.0 - D**2), D)

    theta3 = np.pi - alpha - beta

```

```python
        # ----------------------------------------------------------
        # 5. Shoulder angle
        # ----------------------------------------------------------

        D2 = (la**2 + P**2 - l7**2) / (2 * la * P)

        if D2 < -1.0 or D2 > 1.0:
            return {}, False

        D2 = np.clip(D2, -1.0, 1.0)

        gamma = np.arctan2(
            np.sqrt(1.0 - D2**2),
            D2
        )

        alpha = np.arctan2(Zd, R)

        theta2 = beta - gamma - alpha

        # ----------------------------------------------------------
        # 6. Wrist angle
        # ----------------------------------------------------------

        theta4 = np.pi / 2.0 - theta2 - theta3

        # ----------------------------------------------------------
        # 7. Wrist roll
        # ----------------------------------------------------------

        theta5 = 0.0

        q = {
            "shoulder_pan": theta_1,
            "shoulder_lift": theta2,
            "elbow_flex": theta3,
            "wrist_flex": theta4,
            "wrist_roll": theta5,
        }

        return q, True
```

CSE 4360/5364 Homework 3 - Velocity Kinematics Solution

I used Matlab's symbolic math toolbox to calculate the Jacobian using the geometric approach in the book. I then used matlabs jacobian function to directly calculate it by taking derivatives. I subtract one solution from the other to confirm they are the same.

SO-101 linear-velocity Jacobian for the first four arm joints.

The Jacobian maps

`[q1_dot; q2_dot; q3_dot; q4_dot]`

to the tool-tip linear velocity expressed in the given base frame:

`v_T^B = Jv(q) * [q1_dot; q2_dot; q3_dot; q4_dot]`

Therefore, Jv is a 3-by-4 matrix. Wrist roll q5 does not change the tool-tip position, so q5_dot is not included.

```
clear
clc
syms q1 q2 q3 q4 q5 real
```

Robot dimensions

```
l1 = 0.0388353;
l2 = 0.0624;
l3 = 0.0303992;
l4 = 0.0542;
l5 = 0.11257;
l6 = 0.0280;
l7 = 0.1349;
l8 = 0.0611;
l9 = 0.1004;

La = sqrt(l5^2 + l6^2);
beta = atan2(l5, l6);
```

Homework 2 forward-kinematics transforms

CT defined below

```
H_B0 = CT(0, l2, l1, -sym(pi));
H_01 = CT(q1,               -l4,  l3,  sym(pi)/2);
H_12 = CT(sym(pi) - beta + q2, 0, -La,          0);
H_23 = CT(beta + q3,          0, -l7,          0);
```

```
H_34 = CT(sym(pi)/2 + q4,     0,   0,  sym(pi)/2);
H_4T = CT(q5, -(l8+l9), 0, 0);
```

Base-to-frame transforms

```
H_B1 = H_B0*H_01;
H_B2 = H_B1*H_12;
H_B3 = H_B2*H_23;
H_B4 = H_B3*H_34;
H_BT = H_B4*H_4T;
```

Joint-axis directions and origins, expressed in the base frame

For a revolute joint, the linear-velocity Jacobian column is

`Jv_i = z_(i-1) cross (p_T - p_(i-1)).`

```
z0 = H_B0(1:3,3); p0 = H_B0(1:3,4);
z1 = H_B1(1:3,3); p1 = H_B1(1:3,4);
z2 = H_B2(1:3,3); p2 = H_B2(1:3,4);
z3 = H_B3(1:3,3); p3 = H_B3(1:3,4);
pT = H_BT(1:3,4);
```

Linear-velocity Jacobian using joint axes and origins

```
Jv1 = cross(z0, pT-p0);
Jv2 = cross(z1, pT-p1);
Jv3 = cross(z2, pT-p2);
Jv4 = cross(z3, pT-p3);

%the approach shown in the book
Jv_geom = [Jv1 Jv2 Jv3 Jv4];

%if you take derivatives.  matlab function jacobian will do that
%automatically
Jv_diff = jacobian(pT, [q1 q2 q3 q4]);
```

Display Clean Decimal Results

```
%force_decimal is defined below.  Matlab has an annoying habit of leaving
%terms as ratios of huge floats or not resolving tiny numbers as 0.
fprintf('Tool-tip position pT:\n')
disp(forceDecimal(pT))

fprintf('\nLinear-velocity Jacobian Jv_geom:\n')
disp(forceDecimal(Jv_geom))
```

```
fprintf('\nLinear-velocity Jacobian Jv_diff:\n')
disp(forceDecimal(Jv_diff))

fprintf('\nDifference Jv_cross - Jv_diff (should be 3x4 zeros):\n')
pretty(forceDecimal(Jv_geom - Jv_diff))
```

```
Tool-tip position pT:
```

Standard DH transform function

```
function H = CT(theta, d, a, alpha)
H = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(alpha), a*cos(theta);
    sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(alpha), a*sin(theta);
    0,             sin(alpha),             cos(alpha),            d;
    0,                      0,                      0,            1];
end
```

Helper function: Forces numeric evaluation and simplifies trig terms

I used Gemini to figure this out.

```
function out = forceDecimal(expr)
% 1. Expand products to expose all numbers multiplying each other
e = expand(expr);
% 2. Evaluate all numeric products into 4-digit floating point decimals
e = vpa(e, 4);
% 3. Re-combine sine and cosine terms back into clean angles
out = combine(e, 'sincos');
end
```

```
0.0675*cos(q1 + q2 + q3) + 0.0580*cos(q1 - q2 + 1.3270) + 0.0675*cos(q2 - q1 + q3) +
0.0807*cos(q1 + q2 + q3 + q4) + 0.0304*cos(q1) + 0.0807*cos(q2 - q1 + q3 + q4) + 0.0580*cos(q1 +
q2 - 1.3270) + 0.0388
         0.0580*cos(q1 + q2 + 0.2438) - 0.0675*sin(q1 + q2 + q3) - 0.0580*cos(q2 - q1 + 0.2438) +
0.0675*sin(q2 - q1 + q3) - 0.0807*sin(q1 + q2 + q3 + q4) - 0.0304*sin(q1) + 0.0807*sin(q2 - q1 +
q3 + q4)

0.1160*cos(q2 + 0.2438) - 0.1615*sin(q2 + q3 + q4) - 0.1349*sin(q2 + q3) + 0.1166


Linear-velocity Jacobian Jv_geom:
[  0.0580*cos(q1 + q2 + 0.2438) - 0.0675*sin(q1 + q2 + q3) - 0.0580*cos(q2 - q1 + 0.2438) +
0.0675*sin(q2 - q1 + q3) - 0.0807*sin(q1 + q2 + q3 + q4) - 0.0304*sin(q1) + 0.0807*sin(q2 - q1 +
q3 + q4), 0.0580*cos(q1 + q2 + 0.2438) - 0.0675*sin(q1 + q2 + q3) + 0.0580*cos(q2 - q1 + 0.2438)
- 0.0675*sin(q2 - q1 + q3) - 0.0807*sin(q1 + q2 + q3 + q4) - 0.0807*sin(q2 - q1 + q3 + q4), -
0.0675*sin(q1 + q2 + q3) - 0.0675*sin(q2 - q1 + q3) - 0.0807*sin(q1 + q2 + q3 + q4) -
0.0807*sin(q2 - q1 + q3 + q4), - 0.0807*sin(q1 + q2 + q3 + q4) - 0.0807*sin(q2 - q1 + q3 + q4)]
[- 0.0675*cos(q1 + q2 + q3) - 0.0580*cos(q1 - q2 + 1.3270) - 0.0675*cos(q2 - q1 + q3) -
0.0807*cos(q1 + q2 + q3 + q4) - 0.0304*cos(q1) - 0.0807*cos(q2 - q1 + q3 + q4) - 0.0580*cos(q1 +
q2 - 1.3270), 0.0580*cos(q1 - q2 + 1.3270) - 0.0675*cos(q1 + q2 + q3) + 0.0675*cos(q2 - q1 + q3)
```

```
- 0.0807*cos(q1 + q2 + q3 + q4) + 0.0807*cos(q2 - q1 + q3 + q4) - 0.0580*cos(q1 + q2 - 1.3270),
0.0675*cos(q2 - q1 + q3) - 0.0675*cos(q1 + q2 + q3) - 0.0807*cos(q1 + q2 + q3 + q4) +
0.0807*cos(q2 - q1 + q3 + q4),   0.0807*cos(q2 - q1 + q3 + q4) - 0.0807*cos(q1 + q2 + q3 + q4)]
[
0,
- 0.1615*cos(q2 + q3 + q4) - 0.1160*cos(q2 - 1.3270) - 0.1349*cos(q2 + q3),
- 0.1615*cos(q2 + q3 + q4) - 0.1349*cos(q2 + q3),                                    -
0.1615*cos(q2 + q3 + q4)]


Linear-velocity Jacobian Jv_diff:
[  0.0580*cos(q1 + q2 + 0.2438) - 0.0675*sin(q1 + q2 + q3) - 0.0580*cos(q2 - q1 + 0.2438) +
0.0675*sin(q2 - q1 + q3) - 0.0807*sin(q1 + q2 + q3 + q4) - 0.0304*sin(q1) + 0.0807*sin(q2 - q1 +
q3 + q4), 0.0580*cos(q1 + q2 + 0.2438) - 0.0675*sin(q1 + q2 + q3) + 0.0580*cos(q2 - q1 + 0.2438)
- 0.0675*sin(q2 - q1 + q3) - 0.0807*sin(q1 + q2 + q3 + q4) - 0.0807*sin(q2 - q1 + q3 + q4), -
0.0675*sin(q1 + q2 + q3) - 0.0675*sin(q2 - q1 + q3) - 0.0807*sin(q1 + q2 + q3 + q4) -
0.0807*sin(q2 - q1 + q3 + q4), - 0.0807*sin(q1 + q2 + q3 + q4) - 0.0807*sin(q2 - q1 + q3 + q4)]
[- 0.0675*cos(q1 + q2 + q3) - 0.0580*cos(q1 - q2 + 1.3270) - 0.0675*cos(q2 - q1 + q3) -
0.0807*cos(q1 + q2 + q3 + q4) - 0.0304*cos(q1) - 0.0807*cos(q2 - q1 + q3 + q4) - 0.0580*cos(q1 +
q2 - 1.3270), 0.0580*cos(q1 - q2 + 1.3270) - 0.0675*cos(q1 + q2 + q3) + 0.0675*cos(q2 - q1 + q3)
- 0.0807*cos(q1 + q2 + q3 + q4) + 0.0807*cos(q2 - q1 + q3 + q4) - 0.0580*cos(q1 + q2 - 1.3270),
0.0675*cos(q2 - q1 + q3) - 0.0675*cos(q1 + q2 + q3) - 0.0807*cos(q1 + q2 + q3 + q4) +
0.0807*cos(q2 - q1 + q3 + q4),   0.0807*cos(q2 - q1 + q3 + q4) - 0.0807*cos(q1 + q2 + q3 + q4)]
[
0,
- 0.1615*cos(q2 + q3 + q4) - 0.1160*cos(q2 - 1.3270) - 0.1349*cos(q2 + q3),
- 0.1615*cos(q2 + q3 + q4) - 0.1349*cos(q2 + q3),                                    -
0.1615*cos(q2 + q3 + q4)]


Difference Jv_cross - Jv_diff (should be 3x4 zeros):
/ 0, 0, 0, 0 \
|            |
| 0, 0, 0, 0 |
|            |
\ 0, 0, 0, 0 /
```

Published with MATLAB® R2026a